



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

SISTEMAS DISTRIBUIDOS

LABORATORIO SESIÓN 5

CONTRERAS DEL ÁNGEL DIEGO ADRIÁN

SEMESTRE 2027-1

27 DE SEPTIEMBRE DE 2026

1. DESPLIEGUE Y ORQUESTACIÓN DE LA INFRAESTRUCTURA

Para dar inicio a la práctica de laboratorio, nos posicionamos en el directorio raíz del proyecto Mexi Banco, donde se ubican la definición del modelo de la aplicación y el archivo de orquestación `docker-compose.yml`. A través del motor de Docker, se ejecutó el comando de construcción y levantamiento de servicios en segundo plano (modo desatendido). Este comando desencadenó la construcción secuencial de la imagen multi-etapa de la aplicación Java a partir de la imagen base `eclipse-temurin:21-jdk-alpine` y el posterior despliegue del contenedor de la base de datos relacional PostgreSQL 17.

Una vez completado el proceso de compilación y empaquetamiento, el orquestador verificó de manera automatizada el estado de salud de los servicios mediante los mecanismos de `healthcheck` configurados internamente. Estos controles garantizan que la aplicación Spring Boot no intente establecer conexiones con la base de datos hasta que el motor PostgreSQL haya completado sus rutinas internas de inicio y esté completamente preparado para recibir peticiones transaccionales en el puerto predeterminado.

```
docker compose up --build -d
```

```
#1 [internal] load build definition from Dockerfile
#2 [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk-alpine
#3 [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine
[+] Up 3/3
Image mexi-banco-app   Built
Container mexi-banco-db-1   Healthy
Container mexi-banco-app-1   Running
```

A continuación, se comprobó el estado operacional de los contenedores activos mediante la consulta de la tabla de procesos del orquestador. La inspección confirmó que tanto el contenedor del backend (`mexi-banco-app-1`) como el de la base de datos (`mexi-banco-db-1`) se mantenían en estado *healthy*, validando el correcto aislamiento de la red virtual privada y el enlace entre componentes.

```
docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
mexi-banco-app-1	mexi-banco-app	"java -jar app.jar"	app	5 hours ago	Up 44 minutes (healthy)

mexi-banco-db-1 postgres:17-alpine "docker-entrypoint.s..." db 5 hours ago Up 45 minutes (healthy)

2. VERIFICACIÓN DEL ESTADO DE SALUD DE LA API (ACTUATOR)

Haciendo uso del cliente de pruebas HTTP Postman, se enviaron peticiones dirigidas a los endpoints expuestos por la colección del proyecto para auditar la respuesta del servidor en formato JSON. En primera instancia, se efectuó una consulta hacia el punto final de diagnóstico y salud del sistema con el fin de evaluar las sondas de *liveness* (disponibilidad de ejecución) y *readiness* (preparación para recibir tráfico bancario).

```
{
  "status": "UP",
  "groups": [ "liveness", "readiness" ]
}
```

3. PRUEBAS TRANSACCIONALES Y VERIFICACIÓN DE IDEMPOTENCIA SPEI

Posteriormente, se llevaron a cabo las pruebas relativas a la ejecución de transferencias de fondos entre cuentas bancarias. Un aspecto fundamental evaluado en esta sección fue el cumplimiento estricto de la propiedad de idempotencia en las operaciones interbancarias SPEI. La idempotencia asegura que, ante reintentos de red o envíos duplicados de una misma solicitud por parte del cliente, el sistema procese el débito y acreditación una única vez.

Al realizar peticiones reiteradas utilizando la misma clave única de idempotencia en el encabezado o cuerpo del mensaje, el módulo de SPEI validó el registro previo en la base de datos y retornó la respuesta transaccional almacenada sin alterar los balances. Este mecanismo previene cargos duplicados en la cuenta ordenante y mantiene la integridad patrimonial del usuario.

```
{
  "id": 4,
  "claveOrigen": "002180790138068198",
  "claveDestino": "012180790138068198",
  "monto": 200.00,
  "fecha": "2026-09-23T04:35:34.654084Z"
}
```

Tras la ejecución exitosa de los movimientos transaccionales, se consultó el estado actualizado de la cuenta destino para verificar la correcta actualización de los balances y el almacenamiento

del titular correspondiente en la capa de persistencia relacional.

```
{  
  "id": 10,  
  "clabe": "002180790138269800",  
  "titular": "Ana",  
  "saldo": 750.00  
}
```

4. AUDITORÍA DE HISTORIAL DE MOVIMIENTOS Y NOTIFICACIONES

Se efectuaron peticiones adicionales para obtener la lista histórica de movimientos financieros e historial de avisos emitidos por el sistema. Esta verificación permitió constatar que cada transacción realizada genera un registro contable inmutable asociado a la CLABE de la cuenta, incluyendo el tipo de movimiento, el impacto en el balance (monto negativo o positivo), el saldo resultante post-operación y la referencia bancaria interbancaria.

```
[  
  {  
    "id": 24,  
    "clabeCuenta": "002180790138269800",  
    "tipo": "SPEI_ENVIADO",  
    "monto": -50.00,  
    "saldoResultante": 750.00,  
    "fecha": "2026-09-23T04:48:41.030659Z",  
    "referencia": "BBVA 012180000000000899"  
  }  
]
```

5. MANEJO DE ERRORES Y VALIDACIONES DE REGLAS DE NEGOCIO

Se evaluó el comportamiento de la aplicación ante situaciones anómalas o violaciones a las reglas de negocio bancarias, tales como el intento de transferir un monto superior al saldo disponible en la cuenta de origen. La API REST interceptó adecuadamente la excepción mediante un controlador global de errores, respondiendo con un formato estandarizado RFC 7807 (Problem Details). Se confirmó que la transacción fue abortada (*rollback*), impidiendo saldos negativos no autorizados y manteniendo la consistencia de los datos.

6. ANÁLISIS EXPERIMENTAL DE PERSISTENCIA Y VOLÚMENES DE DOCKER

Uno de los aspectos centrales analizados durante la práctica fue el comportamiento del almacenamiento no volátil en entornos con contenedores mediante la gestión de volúmenes nombrados de Docker Compose. Se probaron dos escenarios de detención del entorno para validar la durabilidad de la información almacenada en PostgreSQL:

- **Escenario A (Reinicio Estándar):** Se procedió a apagar el entorno utilizando la instrucción habitual de apagar servicios. Posteriormente, la infraestructura se volvió a levantar en segundo plano. Dado que el volumen asociado al directorio de datos del motor (/var/lib/postgresql/data) permaneció intacto en el host, la base de datos conservó la totalidad de los registros de cuentas, movimientos y claves de idempotencia previamente creados.

```
docker compose down
```

```
docker compose up -d
```

- **Escenario B (Eliminación Explícita de Volúmenes):** En segundo lugar, se detuvieron los contenedores agregando la bandera -v, solicitando explícitamente la destrucción del volumen administrado. Al volver a iniciar el entorno, la base de datos se inicializó completamente limpia. Al intentar consultar una CLABE previamente existente mediante Postman, la aplicación devolvió un código de estado 404 (Not Found), demostrando experimentalmente que la persistencia depende de la existencia del volumen en el host.

```
docker compose down -v
```

```
docker compose up -d
```

```
{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No existe la CLABE 002180790146117263",
  "instance": "/cuentas/002180790146117263"
}
```

7. ARQUITECTURA DEL MONOLITO Y SEPARACIÓN POR CAPAS

El análisis del código fuente de la aplicación Mexi Banco permitió identificar una estructura monolítica distribuida funcionalmente en los módulos de Cuenta, Movimiento, Transferencia, Notificación y SPEI. El sistema implementa el patrón de diseño en cuatro capas tradicionales:

- **Capa Controlador (REST Controllers):** Encargada de exponer los puntos de entrada HTTP, validar los DTOs de entrada, serializar/deserializar objetos JSON y gestionar los códigos de estado de respuesta.
- **Capa Servicio (Business Logic Layer):** Contiene las reglas del negocio bancario, el manejo explícito de transacciones contables, la verificación de saldo e idempotencia.
- **Capa Repositorio (Data Access Layer - Spring Data JPA):** Provee la abstracción de acceso a datos mediante consultas SQL y mapeos objeto-relacional (ORM).
- **Capa Entidad (Domain Models):** Define la estructura de las tablas en PostgreSQL, anotaciones de llaves primarias, relaciones entre tablas y restricciones de integridad.

8. ESTRATEGIA DE CONSTRUCCIÓN MULTI-ETAPA (DOCKERFILE MULTI-STAGE)

La inspección del archivo Dockerfile evidenció la implementación de la buena práctica de construcción multi-etapa (*multi-stage build*), diseñada para optimizar la seguridad y el tamaño final de la imagen del contenedor:

- **Primera Etapa (Builder):** Utiliza la imagen base eclipse-temurin:21-jdk-alpine equipada con el Kit de Desarrollo de Java completo y Maven. En esta fase se compilan las clases, se descargan las dependencias del proyecto y se genera el archivo ejecutable empaquetado (app.jar).
- **Segunda Etapa (Runtime):** Utiliza una imagen ligera basada únicamente en el entorno de ejecución eclipse-temurin:21-jre-alpine. Se copia únicamente el ejecutable generado en la etapa anterior, descartando las herramientas de compilación, el código fuente y las librerías de Maven. Esta separación reduce drásticamente el peso de la imagen final y disminuye la superficie de ataque frente a vulnerabilidades.

9. CONFIGURACIÓN DEL ENTORNO DE ORQUESTACIÓN CON DOCKER COMPOSE

La revisión del archivo docker-compose.yml hizo posible comprender la articulación de la infraestructura declarativa compuesta por la aplicación Java y PostgreSQL:

- **Persistencia Manejada:** Declaración del volumen nombrado postgres_data para asegurar la supervivencia de los datos contables.
- **Aislamiento de Red:** Creación de una red virtual tipo *bridge* que habilita la resolución DNS interna por nombre de servicio entre contenedores.
- **Orquestación Dependiente de Salud:** Directiva depends_on combinada con condition: service_healthy, garantizando el orden de arranque adecuado y evitando fallos por

conexiones rechazadas durante la fase de inicialización.

10. CONCLUSIÓN

La realización de esta práctica si me generó conflictos, porque hubo partes en las que me perdí o no sabía que realizar porque mi computadora se trababa pero gracias a ella me dió una mejor comprensión de los procesos de empaquetamiento, despliegue y orquestación de sistemas bancarios monolíticos utilizando contenedores de software. Se comprobó en la práctica el impacto que tiene la administración adecuada de volúmenes persistentes para garantizar la integridad de la información en servicios con estado, así como la utilidad de contar con pruebas de idempotencia e interceptores de error en arquitecturas distribuidas.